

Crypto Investigation — Guided Lab

OSINT & Blockchain Forensics — the \$TOK case, step by step

Day 3 · guided · from on-chain identifiers to a full digital-identity + money trail

You investigate a Solana memecoin, **TOKTHEDOG (\$TOK)**, starting from a few on-chain identifiers. You will pivot to its Web2 footprint, trace the money to the deployer and the liquidity, and automate the metadata collection in Python. Every step is guided; the instructor holds a methodology correction (`WriteUp_FR_CryptoInvestigation.md`).

The original case brief is `Case_Brief_original.pdf` in this folder. This document turns it into a followed, step-by-step lab with checks and a scoring grid.

Seed data

Entity	Solana address (partial)
Token Mint (\$TOK)	7fuinNNcyXXXXXXXXXXXXX1BVCnVwoXeVQ4cXP
Pair / Liquidity Pool	E5ihhxqKYnzpXXXXXXXXXXwpQAHmtTKJhKZMwN8v9
Wrapped SOL Mint	So1111111111111111111111111111111111112

Replace the partial addresses with the real ones your instructor provides. The **method** is what is graded, and it transfers to any token.

Learning objectives

- Explain Solana's model: Mint address, ATA, liquidity pool/pair, SPL tokens.
- Pivot from an on-chain identifier to Web2 (Telegram, X, websites) via DEX aggregators.
- Trace money: deployer, funding source, liquidity status, holder clustering.
- Automate metadata + risk collection with Python and public REST APIs.
- Rate every finding (Admiralty) and keep provenance (URL, UTC, hash).

Setup

```
cd "Day_3/Crypto Investigation Lab"
```

```
pip install requests
```

Part 1 — Understand Solana (30 min)

Answer before touching a tool.

1.1 What is the difference between a **Mint address** and a **Pair address**? What does each identify?

1.2 On Solana, a wallet does not hold a token balance directly — what is an **Associated Token Account (ATA)** and why does it exist?

1.3 Why is the **liquidity-creation transaction** the key to finding the original developer (the *deployer*)?

Part 2 — Web2 pivots via DEX aggregators (35 min)

Search the Mint or Pair on **DexScreener**, **GeckoTerminal** or **Birdeye**.

2.1 Extract the token's official description (Metaplex metadata) and **every** linked social handle (Telegram, X, website). Record each with its URL and an Admiralty rating.

2.2 Detect the launch platform: bonding-curve launchpad (e.g. Pump.fun) or a direct DEX pool? What evidence tells you which?

2.3 Social forensics: on Telegram (Telemetrio / TGStat) note the channel creation date and subscriber growth; on X, search the **exact contract address** to find the **first** account that posted it. Why is the first poster significant?

Trap. A token's own site and socials are marketing. Rate them C or below until an independent source confirms. The first on-chain fact outranks any claim on the website.

Part 3 — On-chain traceability (40 min)

Use **Solscan** or **SolanaFM**.

3.1 **Deployer.** Open the Mint page → earliest transaction (Genesis / Mint To) → identify the **signer**. That is the deployer. Record the address.

3.2 Funding source (inflow). Inspect the deployer's earliest SOL transactions. Classify: - Case A — from a **centralised exchange** hot wallet (Binance, Bybit, Coinbase) → a KYC subpoena target. - Case B — from a **mixer / bridge** (FixedFloat, ChangeNOW, Maya) → obfuscation. - Case C — from an **intermediary personal wallet** → keep pivoting. Which case is it, and what is your next step for each?

3.3 Liquidity & holders. Is liquidity **burned** or **locked**? Analyse the **top 10 holders**: any cluster of wallets created within the same minute receiving supply via bundled transactions? What does clustering suggest here?

Part 4 — Automate in Python (30 min)

Start from the reference script pattern (DexScreener REST, keyless):

```
import requests

def analyze(mint):
    r = requests.get(f"https://api.dexscreener.com/latest/dex/tokens/{mint}",
                    headers={"User-Agent": "Mozilla/5.0"}, timeout=20)
    r.raise_for_status()
    pair = r.json()["pairs"][0]
    print(pair["baseToken"]["name"], pair["baseToken"]["symbol"], pair.get("priceUsd"))
    for s in pair.get("info", {}).get("socials", []):
        print("social:", s.get("type"), s.get("url"))
```

4.1 Run it on the Mint. Inspect the raw JSON. Which fields give you the socials, the DEX and the pair address?

4.2 Extend it to query the **RugCheck API** (`https://api.rugcheck.xyz/v1/tokens/{mint}/report`) and print the **risk score**, plus the status of **Mint Authority** and **Freeze Authority**. Why do those two authorities matter for a rug-pull assessment?

4.3 Output a small provenance-carrying table (name, symbol, price, socials, risk, retrieved-at UTC) and save it to CSV.

Deliverables

Submit one structured investigation report:

1. **Digital identity sheet**: verified socials, lore, active trading platforms — each rated.
2. **On-chain genealogy**: deployer → funding source → pool creation, with addresses.
3. **Legitimacy assessment**: liquidity lock/burn + holder-cluster analysis + RugCheck score.
4. **Automation**: your working `.py` and the CSV it produces.

5. Every finding carries an **Admiralty rating** and a **source URL + UTC**.

Grading grid (100 pts)

Item	Pts	What earns the points
Solana concepts	15	Mint vs Pair, ATA, deployer via liquidity tx
Web2 pivots	20	all socials found + rated; launch platform identified
On-chain trace	30	deployer + funding case + liquidity/holder clustering
Automation	25	DexScreener + RugCheck script, CSV with provenance
Rigour	10	Admiralty ratings, provenance, first-poster reasoning

Optional Extensions (Bonus)

- **Genealogy graph**: build the deployer → funding → pool graph in `networkx` and export to Gephi.
- **Emit STIX**: model the token, the deployer wallet and the socials as STIX objects (see the STIX Lab).
- **Cross-chain**: if funds came via a bridge, follow them to the source chain and repeat the trace.